ISTA 130

Conditional Statements I

Last Time

Statements

Expressions

`input` vs `print`

String concatenation

Statements

A **statement** is the smallest executable unit of code (kind of like a sentence).

Examples:

- ➔ `yertle.forward(100)`
- ➔ `square(yertle, 50)`
- ➔ `x = math.sqrt(100)`

Expressions

An **expression** is a combination of literals, variables, operators and functions which evaluates to a single value.

➔ `2 + 2`
➔ `73.0 * (9 + x)`
➔ `600 + math.sqrt(some_number * 2.0)`

`input` vs `print`

`input()`	`print()`
Gets info from the keyboard.	Sends info to the screen.
Zero or one arguments (to print as a prompt).	Zero to infinity arguments (to print as an end in itself).
Returns a string - must cast if numbers are needed.	Returns `None`.
Gets a string from the user and returns it to the calling code.	Prints a string from its arguments.

String concatenation vs `print`

String concatenation	`print()`
Is an operation.	Is a function.
Separate substrings with +	Separate arguments with ,
Put in all spaces	Python adds spaces between arguments
Must cast integers and floats to strings	Don't have to cast arguments.
Is string concatenation	Can use string concatenation in arguments

`print` with multiple arguments vs `print` with string concatenation

String concatenation	Multiple Arguments
Separate substrings with +	Separate arguments with ,
Put in all spaces	Python adds spaces between arguments
Must cast integers and floats to strings	Don't have to cast arguments.
Is string concatenation	Can use string concatenation in arguments

What's new?

Decisions using comparisons - *if* something is the case (condition), do action

Conditions in `python`

- ➔ `if`
- ➔ Comparisons
- ➔ `else`
- ➔ `elif`
- ➔ Logical Operators

Shopping

Imagine your roommate is going shopping for you and you'd like to send your grocery list along

It might look something like this:

```
bread
pickles
spam
```

You really want a pint of Haagen Dazs ice cream but it's pretty expensive
so you decide you only want some if it's on sale
in fact, you only want it if it's less than $3.00

How would you explain that to your roommate?
How would you write it on your grocery list?

Shopping

You might add something like this:

```
bread
pickles
spam
if Haagen Dazs is on sale for less than $3.00
    buy a pint of Haagen Dazs rum raisin
```

When you get home from work you race to the freezer and find no ice cream :(

What can you conclude?

Shopping

The following week your roommate goes shopping again

You still want the Haagen Dazs
- but you can't bear the thought of another week without ice cream
- you'll settle for a cheaper brand if you have to

How would you explain this to your roommate?
How would your write it on your grocery list?

Shopping

You might do something like this:

```
bread
pickles
spam
if Haagen Dazs costs less than $3.00
    buy a pint of Haagen Dazs rum raisin
otherwise
    buy a pint of Blue Bunny vanilla
```

Your roommate will either buy the Haagen Dazs or the Blue Bunny

Assuming your roommate comes home with Blue Bunny:

➔ What do you know about Haagen Dazs?

Shopping

After eating the Blue Bunny, you decide you need to explore other options before being stuck with the Bunny.

You might do something else:

```
bread
pickles
spam
if Haagen Dazs costs less than $3.00
    buy a pint of Haagen Dazs rum raisin
otherwise if Breyers costs less than $3.00
    buy a pint of Breyers strawberry
otherwise
    buy a pint of Blue Bunny vanilla
```

Shopping

```
bread
pickles
spam
if Haagen Dazs costs less than $3.00
    buy a pint of Haagen Dazs rum raisin
otherwise if Breyers costs less than $3.00
    buy a pint of Breyers strawberry
otherwise
    buy a pint of Blue Bunny vanilla
```

Assuming your roommate comes home with Blue Bunny, what do you know about the other ice creams?

If your roommate follows your instructions could you end up with more than one pint of ice cream?

Shopping

Later your change the grocery list to this:

```
bread
pickles
spam
if Haagen Dazs costs less than $3.00
    buy a pint of Haagen Dazs rum raisin
otherwise if Breyers costs less than $3.00
    buy a pint of Breyers strawberry
otherwise if Blue Bell costs less than $3.00
    buy a pint of Blue Bell black walnut
otherwise if ...
    ...the list goes on and on for several pages...
otherwise
    buy a pint of Blue Bunny vanilla
```

If your roommate finds Haagen Dazs on sale for $2.50 per pint, will your roommate read the rest of the instructions?

Shopping

```
bread
pickles
spam
if Haagen Dazs costs less than $3.00
     buy a pint of Haagen Dazs rum raisin
otherwise if Breyers costs less than $3.00
     buy a pint of Breyers strawberry
otherwise if Blue Bell costs less than $3.00
     buy a pint of Blue Bell black walnut
otherwise if ...
     ...the list goes on and on for several pages...
otherwise
     buy a pint of Blue Bunny vanilla
```

Imagine someone in the store bumps into your roommate

- ➔ Your roommate puts a finger on the grocery list to mark their reading position
- ➔ The finger is in the middle of the line that says "otherwise if Dreyers..."
- ➔ What can you tell me about the other ice creams?

Shopping

Let's go back to an earlier shopping list:

Would you ever write a shopping list like this? Why not?

```
if Haagen Dazs costs less than $3.00
    bread
    pickles
    spam
    mustard
    pint of Haagen Dazs rum raisin
otherwise
    bread
    pickles
    spam
    mustard
    pint of Blue Bunny vanilla
```

Shopping

It makes more sense like this:

```
bread
pickles
spam
mustard
if Haagen Dazs costs less than $3.00
    buy a pint of Haagen Dazs rum raisin
otherwise
    buy a pint of Blue Bunny vanilla
```

Why?

➔ The ice cream purchased depends on the ice cream price

➔ Nothing else does, we want our spam and pickle sandwiches (gross) regardless of ice cream prices.

She's a witch!



Conditional Execution

Python's *conditional statement*

```
if
```

Its use looks similar to how we might say something in English:

```
if condition:
    statement(s) to execute
```

For example:

```
if price < 3.00:
    print('Buy the Haagen Dazs!')
```

Conditional Execution

```
if price < 3.00:
    print('Buy the Haagen Dazs!')
```

This will *only* print "Buy the Haagen Dazs!" *if* the value that **`price`** refers to is smaller than 3.00

- ➔ Execution of the **`print`** statement is *conditional*
- ➔ It only happens when the *condition* is `True`
  - ◆ i.e., only if the `price` *is indeed* strictly less than 3.00
- ➔ If not, the **`print`** statement is skipped and Python moves on.

Compound Statements

```
if price < 3.00:
    print('Buy the Haagen Dazs!')
    print('Yipppeeeeee!!!')
```

Notice that this statement is similar to `for`-loops and function definitions.

➔ Each has a *header* line ending in a colon and an indented *body* section.

Statements like these are called ***compound statements***

➔ They are statements that contain other statements

➔ Like `for`-loops and function definitions, you can have as many statements inside of the compound statement as you need, they just have to be indented

Compound Statements

```
if price < 3.00:
    print('Buy the Haagen Dazs!')
    print('Yipppeeeeee!!!')
print('I love ice cream!')
```

Like `for` loops and functions, indented code tells Python what code is part of the compound statement

- ➔ In the example above, the phrase "I like ice cream!" will definitely be printed
- ➔ Only the two indented print statements are contained within the `if` statement
  - ◆ They will only print if `price` was previously set to a value smaller than 3.00

Comparison Operators

In order to make interesting conditions we have a set of *comparison* operators:

- `<` → "less than"
- `>` → "greater than"
- `<=` → "less than or equal to"
- `>=` → "greater than or equal to"
- `==` → "equal to"
- `!=` → "*not* equal to"

What is =?

- Note that these are meaningless:
  - `=>`
  - `=<`

Boolean Expressions

We've seen *expressions* that evaluate to `int`, `float`, and `str` types:

- ➔ `10 + 5 → type is` **`int`**
- ➔ `6.0 * 20 → type is` **`float`**
- ➔ `'X' * 5 → type is` **`str`**

The conditions we use in our `if` statements evaluate to a new type called `bool`

- ➔ The `bool` type has only two values: `True`, `False`
- ➔ Note: these are *not* strings! Try `type(True)`.

Boolean Expressions

Expressions that evaluate to the `bool` type are called ***boolean expressions***:

- ➔ `ounces == 20` → `type is` **`bool`**
  - `ounces` is either equal to 20 or it's not
  - It is either `True` that `ounces == 20` or it is `False`
  - It depends on what value `ounces` refers to at the time the expression is evaluated
- ➔ `height_in_feet > 6.0` → `type is` **`bool`**
  - `height_in_feet > 6.0` is either `True` or `False`

Boolean expressions are <u>not</u> if statements! `if` statements contain Boolean expressions.

Comparison Operators

```
if age < 21:
    print('Sorry bub, no beer for you!')

if height_in_feet > 6.0:
    print('Wow, that\'s tall.')

if age <= 14:
    print('Can\'t drive legally yet')

if score >= 90.0:
    print('grade is an A')

if ounces == 20:
    print('Yum, a British Pint!')

if marbles != 10:
    print('I\'ve lost some of my marbles...')
```

What are the boolean expressions in each of these?

Comparison Operators

You can also compare strings:

```
if alien == 'Borg':
    print('Resistance is futile!')

if input != password:
    print('Access Denied!')

if word1 < word2:
    print(word1, 'comes before', word2)
```

Note: upper- and lower-case characters compare differently

- ➔ `'a' < 'b'` → `True`
- ➔ `'a' < 'B'` → `False`
- ➔ Later in the semester we will learn how to deal with this

"Otherwise" in Python

What about our fall-back ice cream plan?

We extend the conditional statement by adding an **`else`**:

```
if condition:
    statement(s)
else:
    statement(s)
```

`if` WITH `else`

```
price = 3.00
if price < 3.00:
    print('Buy the Haagen Dazs!')
    print('Yipppeeeeee!!!')
else:
    print('Buy the Blue Bunny')
    print(':(')
print('I love ice cream!')
```

- When this code is executed Python:
  - reads this expression: `price < 3.00` as: `3.00 < 3.00`
  - evaluates `3.00 < 3.00` finding that it is False
  - skips the body of the `if`, and moves on to the `else`
  - enters the `else` block
    - prints `Buy the Blue Bunny`
    - prints `:(`
  - prints `I love ice cream!`

Decisions with many options

We extend the conditional statement:

- **if**
- **elif**
- **else**

(Note: elif != )

Decisions with many options

```
if condition:
    statement(s)
elif condition:
    statement(s)
elif condition:    # as many elif's as you need
    statement(s)
else:    # deal with the leftovers (anything else)
    statement(s)
```

Decisions with many options

```
if candy_color == 'red':
    print('Cherry')
elif candy_color == 'green':
    print('Lime')
elif candy_color == 'yellow':
    print('Lemon')
elif candy_color == 'orange':
    print('Orange')
else:
    print('No clue!')
```

- Note: only **one** of these print statements will execute
  - Python enters the first condition that evaluates to True, executes the body of that compound statement, and skips all the rest
    - (just like your roommate did with the grocery list)